Pandas Notes: Basics to Intermediate

Transformed from a Kaggle Notebook
Sohamdip Santra

October 5, 2026

Contents

1 Introduction **3**
1.1 Input and Output . . . 3

2 Series **3**
2.1 Creating a Series . . . 4
2.2 Series Attributes and Methods . . . 4

3 DataFrame **5**
3.1 Creating a DataFrame . . . 5
3.2 DataFrame Attributes and Methods . . . 5

4 Indexing and Selection **6**
4.1 Selecting Columns . . . 6
4.2 Adding and Dropping Columns . . . 6
4.3 `loc`, `iloc`, `at`, `iat` . . . 7
4.4 Boolean Indexing and Conditional Selection . . . 7

5 Handling Missing Values **7**

6 Merging, Joining, and Concatenation **8**
6.1 Merge . . . 8
6.2 Join . . . 9
6.3 Concatenation . . . 9

7 GroupBy and Aggregation **9**
7.1 Aggregation . . . 10

8 Pivot Tables and Crosstab **10**
8.1 Pivot Table . . . 10
8.2 Crosstab . . . 10
8.3 Reshaping . . . 10

9 Operations and Apply **11**
9.1 Vectorized Operations . . . 11
9.2 Apply, Map, ApplyMap . . . 11
9.3 String and Datetime Accessors . . . 11

10 Feature Extraction Project: Anime Dataset **11**

11 Countries Dataset Project **13**
11.1 Questions and Solutions 13

12 Function Reference **14**
12.1 Inspection and Summary 14
12.2 Cleaning and Transformation 14
12.3 Combining and Reshaping 14
12.4 Time Series and Strings 15

13 Best Practices and Common Errors **15**

14 Citation **16**

1 Introduction

Pandas is a Python library for data manipulation and analysis. It is built on top of NumPy and provides two main data structures:

- **Series**: a one-dimensional labelled array.
- **DataFrame**: a two-dimensional labelled table, like a spreadsheet or SQL table.

Typical imports:

```
import numpy as np
import pandas as pd
import os
import kagglehub
```

1.1 Input and Output

Common I/O functions:

Function	Purpose
`pd.read_csv()`	Read a CSV file into a DataFrame.
`pd.read_excel()`	Read an Excel file.
`pd.read_json()`	Read JSON.
`pd.read_sql()`	Read from a SQL database.
`df.to_csv()`	Write a DataFrame to CSV.
`df.to_excel()`	Write to Excel.
`df.to_json()`	Write to JSON.

Useful `read_csv` parameters:

```
pd.read_csv(
    "file.csv",
    sep=",", # delimiter
    header=0, # row number to use as column names
    names=["a", "b"], # custom column names
    index_col="id", # column to use as row index
    usecols=["a", "b"], # only read these columns
    dtype={"a": int}, # force column dtypes
    parse_dates=["date"],# parse date columns
    na_values=["NA", "?"],
    nrows=1000,
    skiprows=5,
    encoding="utf-8"
)
```

2 Series

A `Series` is a one-dimensional labelled array. It can be created from a list, NumPy array, dictionary, or scalar.

2.1 Creating a Series

```
labels = ['a', 'b', 'c', 'd']
my_list = [1, 2, 3, 4]
arr = np.array(my_list)
d = {1: 10, 2: 20, 3: 30}

pd.Series(my_list)
pd.Series(my_list, index=labels)
pd.Series(d)
pd.Series(arr)
```

Example output:

```
a 1
b 2
c 3
d 4
dtype: int64
```

2.2 Series Attributes and Methods

Attribute / Method	Description
`s.values`	NumPy array of values.
`s.index`	Index labels.
`s.dtype`	Data type.
`s.name`	Name of the Series.
`s.shape`	Shape tuple.
`s.size`	Number of elements.
`s.head(n)`	First n rows.
`s.tail(n)`	Last n rows.
`s.describe()`	Summary statistics.
`s.value_counts()`	Frequency of unique values.
`s.unique()`	Unique values.
`s.nunique()`	Number of unique values.
`s.sort_values()`	Sort by values.
`s.sort_index()`	Sort by index.
`s.isna()`	Boolean mask of missing values.
`s.notna()`	Boolean mask of non-missing values.
`s.fillna(x)`	Replace missing values with x.
`s.dropna()`	Remove missing values.
`s.apply(func)`	Apply a function elementwise.
`s.map(func)`	Map values using a function or dictionary.
`s.replace(a, b)`	Replace a with b.
`s.astype(type)`	Convert dtype.

3 DataFrame

A `DataFrame` is a two-dimensional labelled table. Each column is a Series.

3.1 Creating a DataFrame

From a dictionary of lists:

```
data = {
    'Name': ['John', 'Anna', 'Peter', 'Linda'],
    'Age': [28, 34, 29, 42],
    'City': ['New York', 'Paris', 'Berlin', 'London'],
    'Salary': [65000, 70000, 62000, 85000]
}
df = pd.DataFrame(data)
df
```

From a list of lists with custom columns and index:

```
data_list = [
    ['John', 28, 'New York', 65000],
    ['Anna', 34, 'Paris', 70000],
    ['Peter', 29, 'Berlin', 62000],
    ['Linda', 42, 'London', 85000]
]

columns = ['name', 'age', 'city', 'salary']
index = ['a', 'b', 'c', 'd']

df2 = pd.DataFrame(data_list, columns=columns)
df3 = pd.DataFrame(data_list, index=index, columns=columns)
df3
```

0utput

	name	age	city	salary
a	John	28	New York	65000
b	Anna	34	Paris	70000
c	Peter	29	Berlin	62000
d	Linda	42	London	85000

3.2 DataFrame Attributes and Methods

Attribute / Method	Description
`df.shape`	Tuple of rows and columns.
`df.columns`	Column labels.
`df.index`	Row labels.
`df.dtypes`	Data type of each column.
`df.info()`	Summary of dtypes, non-null counts, memory.
`df.describe()`	Summary statistics for numeric columns.
`df.head(n)`	First n rows.
`df.tail(n)`	Last n rows.

`df.sample(n)`	Random sample of rows.
`df.T`	Transpose.
`df.sort_values(by=...)`	Sort rows by column values.
`df.sort_index()`	Sort by index.
`df.rename(columns=...)`	Rename columns or index.
`df.reset_index()`	Convert index to column.
`df.set_index()`	Set a column as index.
`df.drop()`	Drop rows or columns.
`df.drop_duplicates()`	Remove duplicate rows.
`df.duplicated()`	Boolean mask of duplicates.
`df.astype()`	Convert dtypes.
`df.replace()`	Replace values.
`df.isin()`	Check membership.
`df.between(a,b)`	Check if values are between a and b.
`df.clip(lower, upper)`	Limit values to a range.
`df.round(n)`	Round numeric values.
`df.corr()`	Correlation matrix.
`df.cov()`	Covariance matrix.
`df.memory_usage(deep=True)`	Memory usage.

4 Indexing and Selection

4.1 Selecting Columns

```
# single column -> Series
names = df3['name']

# multiple columns -> DataFrame
col = ['name', 'salary', 'age']
name_sal = df3[col]
```

4.2 Adding and Dropping Columns

```
df3['Stream'] = [
    'AI-Engineer',
    'Data Scientist',
    'Robotics Engineer',
    'Full Stack Developer'
]

# Drop a column
df3.drop('Stream', axis=1)

# Modify original DataFrame
df3.drop('Stream', axis=1, inplace=True)
```

Axis convention:

- `axis=0`: row-wise operation (default for many methods).

- `axis=1`: column-wise operation.

4.3 `loc`, `iloc`, `at`, `iat`

Indexer	Description
`df.loc[rows, cols]`	Label-based indexing. Inclusive of endpoints.
`df.iloc[rows, cols]`	Integer-position-based indexing. Exclusive of endpoints.
`df.at[row, col]`	Fast scalar label access.
`df.iat[row, col]`	Fast scalar integer-position access.

Examples:

```
df.loc[[1, 2]]
df.loc[[0, 3]][['Name', 'Salary']]

df.iloc[0:2, 0:2]
df.at[0, 'Name']
df.iat[0, 0]
```

4.4 Boolean Indexing and Conditional Selection

```
# Salary greater than 65K
df[df['Salary'] > 65000]

# Multiple conditions: use parentheses
df[(df['Salary'] > 65000) & (df['City'] == 'Paris')]

# OR condition
df[(df['Salary'] > 80000) | (df['City'] == 'Berlin')]

# Using query()
df.query('Salary > 65000 and City == "Paris"')
```

5 Handling Missing Values

Missing values are represented as `NaN` (float) or `None`/`pd.NA`.

```
data = {
    'A': [1, 2, np.nan, 4, 5],
    'B': [1, 2, 3, 4, 5],
    'C': [1, 2, 3, np.nan, np.nan],
    'D': [1, np.nan, np.nan, np.nan, 5]
}
df = pd.DataFrame(data)
df
```

Method	Description
`df.isna()`	Boolean DataFrame of missing values.
`df.isna().sum()`	Count missing values per column.
`df.isna().any()`	Whether any missing value exists per column.
`df.dropna()`	Drop rows with any missing values.
`df.dropna(thresh=3)`	Keep rows with at least 3 non-NA values.
`df.fillna(value)`	Fill missing values.
`df.fillna(df.mean())`	Fill with column means.
`df.ffill()`	Forward fill.
`df.bfill()`	Backward fill.
`df.interpolate()`	Interpolate missing values.

Examples:

```
df.isna()
df.isna().sum()
df.isna().any()
df.dropna()
df.dropna(thresh=3)

df.fillna('pong')
vals = {'A': -1, 'B': -2, 'C': -3, 'D': -4}
df.fillna(value=vals)
df.fillna(df.mean())
```

6 Merging, Joining, and Concatenation

6.1 Merge

`pd.merge()` combines DataFrames using common columns, similar to SQL joins.

```
employees = pd.DataFrame({
    'employee_id': [1, 2, 3, 4, 5],
    'name': ['John', 'Anna', 'Peter', 'Linda', 'Bob'],
    'department': ['HR', 'IT', 'Finance', 'IT', 'HR']
})

salaries = pd.DataFrame({
    'employee_id': [1, 2, 3, 6, 7],
    'salary': [60000, 80000, 65000, 70000, 90000],
    'bonus': [5000, 10000, 7000, 8000, 12000]
})

pd.merge(employees, salaries)
pd.merge(employees, salaries, on='employee_id')
pd.merge(employees, salaries, on='employee_id', how='outer')
pd.merge(employees, salaries, on='employee_id', how='right')
```

Join types:

- `inner`: only matching keys.

- `left`: all keys from left DataFrame.
- `right`: all keys from right DataFrame.
- `outer`: all keys from both.

Additional parameters: `left_on`, `right_on`, `suffixes`, `indicator`, `validate`.

6.2 Join

`df.join()` combines DataFrames on their index by default.

```
df1 = pd.DataFrame({'name': ['Alice', 'Bob', 'Charlie']}, index=[1, 2, 3])
df2 = pd.DataFrame({'score': [85, 90, 75]}, index=[2, 3, 4])

df1.join(df2, how='outer')
```

6.3 Concatenation

`pd.concat()` stacks DataFrames along an axis.

```
df1 = pd.DataFrame({
    'A': ['A0', 'A1', 'A2'],
    'B': ['B0', 'B1', 'B2'],
    'C': ['C0', 'C1', 'C2']
})

df2 = pd.DataFrame({
    'A': ['A3', 'A4', 'A5'],
    'B': ['B3', 'B4', 'B5'],
    'C': ['C3', 'C4', 'C5']
})

pd.concat([df1, df2]) # stack rows
pd.concat([df1, df2], axis=1) # stack columns
```

7 GroupBy and Aggregation

GroupBy follows the split–apply–combine pattern.

```
data = {
    'Category': ['A', 'B', 'A', 'B', 'A', 'B', 'A', 'B'],
    'Store': ['S1', 'S1', 'S2', 'S2', 'S1', 'S2', 'S2', 'S1'],
    'Sales': [100, 200, 150, 250, 120, 180, 200, 300],
    'Quantity': [10, 15, 12, 18, 8, 20, 15, 25],
    'Date': pd.date_range('2023-01-01', periods=8)
}
df = pd.DataFrame(data)

cat = df.groupby(['Store', 'Category'])[['Sales', 'Quantity']].sum()
print(cat)
```

7.1 Aggregation

```
cat['Mean'] = df.groupby(['Store', 'Category'])[['Sales']].mean()
cat['std'] = df.groupby(['Store', 'Category'])[['Sales']].std()

new_cat = cat['Sales'].agg(['sum', 'mean', 'median', 'max', 'min', 'count', 'std'])
bat = df['Sales'].agg(['sum', 'mean', 'median', 'max', 'min', 'count', 'std'])
```

Useful aggregation functions: `sum`, `mean`, `median`, `max`, `min`, `count`, `std`, `var`, `first`, `last`, `nunique`.

Named aggregation:

```
df.groupby('Category').agg(
    total_sales=('Sales', 'sum'),
    avg_sales=('Sales', 'mean'),
    max_quantity=('Quantity', 'max')
)
```

8 Pivot Tables and Crosstab

8.1 Pivot Table

`pd.pivot_table()` creates a spreadsheet-style pivot table. Default aggregation is mean.

```
pd.pivot_table(
    df,
    values=['Sales', 'Units'],
    index='Quarter',
    columns='Product',
    aggfunc='sum'
)
```

8.2 Crosstab

`pd.crosstab()` computes frequency tables.

```
pd.crosstab(df['Region'], df['Rep'])
```

8.3 Reshaping

Function	Description
`df.melt()`	Wide to long format.
`df.pivot()`	Long to wide format.
`df.stack()`	Pivot columns to rows.
`df.unstack()`	Pivot rows to columns.
`df.explode()`	Expand list-like values.
`pd.get_dummies()`	One-hot encoding.
`pd.cut()`	Bin continuous values.

`pd.qcut()`	Quantile-based binning.

9 Operations and Apply

9.1 Vectorized Operations

```
df['A'] * 10 + 2
df['A'].apply(lambda x: x**3 - 3)
df['D'] = df['A'].apply(lambda x: x**3 - 3)
```

9.2 Apply, Map, ApplyMap

Method	Description
`Series.apply(func)`	Apply function elementwise.
`Series.map(func/dict)`	Map values.
`DataFrame.apply(func, axis=0/1)`	Apply along rows or columns.
`DataFrame.applymap(func)`	Apply elementwise (deprecated in newer pandas; use `DataFrame.map`).
`DataFrame.map(func)`	Elementwise mapping.
`DataFrame.pipe(func)`	Chain functions.
`DataFrame.assign(new=...)`	Create new columns.
`DataFrame.eval(expr)`	Evaluate string expression.
`DataFrame.query(expr)`	Query rows using string expression.

9.3 String and Datetime Accessors

```
# String methods
df['Name'].str.upper()
df['Name'].str.lower()
df['Name'].str.contains('a')
df['Name'].str.replace('a', 'A')
df['Name'].str.split(' ')
df['Name'].str.len()

# Datetime methods
df['Date'] = pd.to_datetime(df['Date'])
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month_name()
df['Quarter'] = 'Q' + df['Date'].dt.quarter.astype(str)
df['Day'] = df['Date'].dt.day_name()
```

10 Feature Extraction Project: Anime Dataset

Goal: extract episode count, timestamp, months aired, and answer questions.

```
df = pd.read_csv('/kaggle/input/datasets/abhigangwar30/anime-project/anime.csv')

def exct_epi(title):
    if '(' in title:
        start = title.index('(')
        if ')' in title and title.index(')') > start:
            epi_count = title[start+1:title.index(')')]
            epi_count = epi_count.split(' ')
            return int(epi_count[0])

def exct_ts(title):
    start = title.index(')') + 1
    ts = title[start:start+19]
    return ts

mon = {
    'Jan':1, 'Feb':2, 'Mar':3, 'Apr':4, 'May':5, 'Jun':6,
    'Jul':7, 'Aug':8, 'Sep':9, 'Oct':10, 'Nov':11, 'Dec':12
}

def mon_cnt(colm):
    colm = colm.split(' - ')
    m1, y1 = colm[0].split(' ')
    m2, y2 = colm[1].split(' ')
    cnt = 0
    if int(y1) < int(y2):
        cnt = int(mon[m2]) + (12 - int(mon[m1]) + 1)
    elif int(y1) == int(y2):
        cnt = int(mon[m2]) - int(mon[m1]) + 1
    else:
        return "NULL"
    return cnt
```

Apply extraction:

```
df["EPISODES"] = [exct_epi(df.loc[i]['Title']) for i in range(df.shape[0])]
df['Time-Stamp'] = df['Title'].apply(exct_ts)
df['Months-Aired'] = df['Time-Stamp'].apply(mon_cnt)
```

Questions answered:

```
# Highest score
df[df['Score'] == df['Score'].max()]['Title']

# Top 5 highest scoring anime
df.sort_values(by='Score', ascending=False)['Title'].head(5)

# Highest episode count
df[df['EPISODES'] == df['EPISODES'].max()][['Title', 'EPISODES']]

# Top 5 episode counts
df.sort_values(by='EPISODES', ascending=False)['Title'].head(5)

# Longest running anime
df[df['Months-Aired'] == df['Months-Aired'].max()]
```

11 Countries Dataset Project

Load data:

```
df = pd.read_csv('/kaggle/input/datasets/krishnaabisht09/countries-dataset/Countries.csv'
    )
df.shape
df.info()
df.describe()
```

Data preprocessing: check missing values, dtypes, and summary statistics.

11.1 Questions and Solutions

```
# Country with highest population and its capital
df.sort_values(by='population', ascending=False).head(1)[['country', 'capital_city', '
    population']]

# Country with least population and its capital
df.sort_values(by='population', ascending=True).head(1)[['country', 'capital_city', '
    population']]

# Top 5 countries by democracy score
df.sort_values(by='democracy_score', ascending=False).head(5)[['country', '
    democracy_score']]

# Number of unique regions
df['region'].unique().shape[0]
df['region'].value_counts().count()

# Countries in Eastern Europe
df[df['region'] == 'Eastern Europe'][['country']]

# Political leader of second highest populated country
df.sort_values(by='population', ascending=False).head(2).tail(1)[['country', '
    political_leader']]

# Countries with unknown political leader
df[df['political_leader'].isna()][['country']]

# Count countries with "Republic" in country_long
def is_republic(name):
    return 'republic' in name.lower()

count = sum(is_republic(i) for i in df['country_long'])
print(count)

# Highest population in Africa
df[df['continent'] == 'Africa'].sort_values(by='population', ascending=False).head(1)[['
    country', 'population']]
```

12 Function Reference

12.1 Inspection and Summary

Function	Description
`df.head(n)`	First n rows.
`df.tail(n)`	Last n rows.
`df.sample(n)`	Random sample.
`df.info()`	dtypes, non-null counts, memory.
`df.describe()`	Numeric summary.
`df.describe(include='all')`	Include categorical columns.
`df.shape`	Dimensions.
`df.columns`	Column names.
`df.index`	Row index.
`df.dtypes`	Data types.
`df.nunique()`	Unique counts.
`df.value_counts()`	Frequency counts.

12.2 Cleaning and Transformation

Function	Description
`df.drop()`	Drop rows/columns.
`df.rename()`	Rename labels.
`df.astype()`	Convert dtype.
`df.replace()`	Replace values.
`df.fillna()`	Fill missing values.
`df.dropna()`	Drop missing values.
`df.drop_duplicates()`	Remove duplicates.
`df.reset_index()`	Reset index.
`df.set_index()`	Set index.
`df.sort_values()`	Sort rows.
`df.sort_index()`	Sort index.
`df.clip()`	Limit values.
`df.round()`	Round values.

12.3 Combining and Reshaping

Function	Description
`pd.merge()`	SQL-style join.
`df.join()`	Index-based join.
`pd.concat()`	Stack DataFrames.
`df.groupby()`	Split-apply-combine.
`df.agg()`	Aggregate.
`df.transform()`	Transform per group.

`df.filter()`	Filter groups.
`pd.pivot_table()`	Pivot table.
`pd.crosstab()`	Frequency table.
`df.melt()`	Wide to long.
`df.pivot()`	Long to wide.
`df.stack()`	Columns to rows.
`df.unstack()`	Rows to columns.
`pd.get_dummies()`	One-hot encoding.

12.4 Time Series and Strings

Function	Description
`pd.to_datetime()`	Convert to datetime.
`pd.date_range()`	Generate date range.
`df['date'].dt.year`	Extract year.
`df['date'].dt.month`	Extract month.
`df['date'].dt.day_name()`	Day name.
`df['date'].dt.quarter`	Quarter.
`df['text'].str.upper()`	Uppercase.
`df['text'].str.contains()`	Substring check.
`df['text'].str.replace()`	Replace substring.
`df['text'].str.split()`	Split string.
`df['text'].str.len()`	String length.

13 Best Practices and Common Errors

- Prefer `df.loc` and `df.iloc` over chained indexing to avoid `SettingWithCopyWarning`.
- Use `inplace=True` carefully; it modifies the original object and returns `None`. Assigning the result is often clearer.
- Remember `axis=0` is row-wise and `axis=1` is column-wise.
- `loc` is label-based and inclusive; `iloc` is position-based and exclusive.
- Use parentheses around each condition in boolean indexing: `(cond1) & (cond2)`.
- Check missing values with `df.isna().sum()` before analysis.
- Use `df.info()` to inspect dtypes and non-null counts.
- For large data, specify `dtype` and `usecols` in `read_csv`.
- Use `df.copy()` when you want to modify a subset without affecting the original.
- Prefer vectorized operations over Python loops for performance.

14 Citation

This notes is made with the reference of my Kaggle notebook while learning Pandas from Sheryians AI School .
